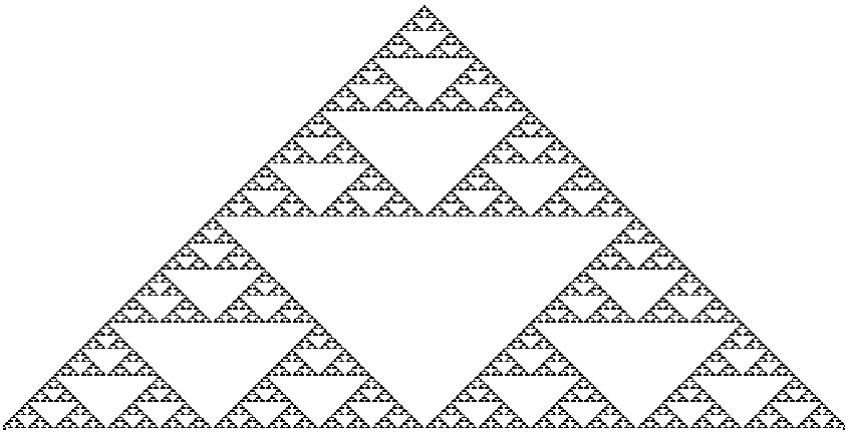
UPC - Universitat Politècnica de Catalunya

Facultat d'Informàtica de Barcelona - FIB

Grau en Intel·ligència artificial

OPT - Optimització

Treballant amb CA i heurístiques



Max Villalba Pons (41684491G)

i

Biel Sánchez Mares (49240671V)

Grup 11

Índex

Primera Sessió **2**
Introducció i objectius 2
Assumpcions del model 2
Modelització en Python 3
Resultats 4
Conclusions 6
Segona Sessió **6**
Introducció i objectius 6
Assumpcions del model 6
Capes del model 7
Model SDL 9
Estat Inicial 10
Estat Unburned 11
Estat Burning 12
Estat Burned 12
Modelització en Python 13
Resultats 14
Conclusions 15
Ús de la IA **15**
Annexos **16**
Primera Sessió 16
Segona Sessió 18

Primera Sessió

Introducció i objectius

Durant la primera sessió, implementarem en Python un autòmat cel·lular capaç de generar les 256 regles de Wolfram, basant-nos en el veïnatge immediat d'una cel·la concreta (esquerra, centre, dreta). Per fer-ho, definirem una condició de frontera per tractar les cel·les que es troben els extrems i així evitar una possible pèrdua d'informació.

Un cop fet això, ens centrarem en explorar una modelització de "gra gruixut" (K = 2), creant una versió simplificada de l'autòmat original a partir d'agrupar blocs en una sola supercel·la.

Finalment, generarem diversos gràfics per tal d'analitzar els resultats i comprovar si el nostre model funciona de manera adequada.

Assumpcions del model

Per treballar de manera adequada, hem hagut d'assumir certes propietats que ens han permès simplificar el modelatge. A continuació definim totes aquestes assumpcions:

- **Treballem en un espai discret i finit:** assumim que en tot moment l'autòmat treballa en un vector d'una dimensió, amb un nombre de cel·les finites definides per l'usuari.

- **Simultaneïtat:** totes les cel·les s'actualitzen de manera simultània en cada generació, és a dir, en cada "pas".

- **Funcionament determinista:** assumirem que l'evolució d'una cel·la depèn exclusivament del seu estat actual i l'estat dels seus veïns, segons la taula de regles definida per la regla escollida.

- **La regla de majoria és suficient:** considerarem que, pel cas del model de "gra gruixut", la regla de la majoria és suficient per representar l'estat d'un bloc. Assumirem que l'empat es resoldrà a favor de l'estat actiu (1).

- **Frontera periòdica:** pels casos extrems assumim que la frontera és periòdica. És a dir, el veí de l'esquerra de la primera cel·la és l'última cel·la, i viceversa.

- **Estat binari:** per tota cel·la, considerem que únicament poden tenir dos estats possibles: 0 o 1.

Modelització en Python

Un cop definides les assumpcions, passarem a l'explicació de com hem modelat aquest problema en Python. Degut a que el propi codi ja està prou comentat en el mateix arxiu Python, ens limitarem a explicar aquelles parts més rellevants per dur a terme aquesta primera sessió.

La lògica del programa està dividida en tres parts diferenciades: una apartat un definim una sèrie de funcions base, un apartat dedicat al "gra gruixut" i finalment un apartat de definició de funcions de visualització i execució principal del programa.

Primerament, si mirem l'apartat de les funcions base, podem veure que definim la lògica essencial de tot el procés. Tenim un funció dedicada a generar la taula de regles (guardant el resultat en un diccionari), així com una funció que ens retorna el veïnat d'una cel·la concreta. En aquesta darrera funció podem veure clarament com s'aplica la frontera periòdica que hem assumit: l'operació mòdul (% n) ens permet "doblegar" l'espai com si fos un anell. A continuació, tenim una funció que defineix l'evolució, és a dir, quina és la generació següent donat un estat i la taula de regles. Finalement, ens trobem amb la funció que executa l'autòmat aplicant les funcions prèviament definides. Aquesta execució retorna l'historial amb totes les generacions fetes.

Tal i com hem comentat abans, el segon apartat d'aquest codi es centra en el "gra gruixut". Per fer-ho, hem considerat dos enfocaments diferents: per una banda, hem decidit aplicar un gra gruixut *a posteriori*, és a dir, aplicar gra gruixut a partir de l'evolució / historial d'un autòmat cel·lular "bàsic" prèviament simulat, aplicant la regla de la majoria a cada bloc de K cel·les contigües; i per altra banda, hem decidit contruïr de zero una taula de regles per al gra gruixut, expandint i projectant les diferents supracel·les (considerant la regla de la majoria), per després simular un autòmat cel·lular de gra gruixut també de zero sense partir d'historials previs. Aquests dos enfocaments ens permetran comparar més endavant els resultats i veure si són equivalents o no.

Finalment, el codi acaba amb múltiples funcions de visualització: visualitzar una sola regla, comparar l'evolució original vs les dos versions de gra gruixut, veure diferents regles en una mateixa graella i combinar múltiples regles alternant generació a generació.

Al final de l'arxiu veiem la lògica d'execució del programa. Cal destacar que al codi hem proposat diversos exemples de visualització, però es poden afegir o treure en funció del que es vulgui visualitzar.

Resultats

Anem ara a analitzar els exemples que hem proposat. Tal i com hem comentat abans, es poden provar tants casos com vulguem afegint les funcions corresponents. Nosaltres hem decidit aquestes per simplificar l'anàlisi.

Els primers tres casos es centren en regles individuals, concretament la regla 30, la 110 i la 90. Aquests experiments ens han permés veure que, efectivament, el nostre codi funciona de manera esperada generant les diferents regles de manera correcta (a l'Annex estan indicades les altres imatges).

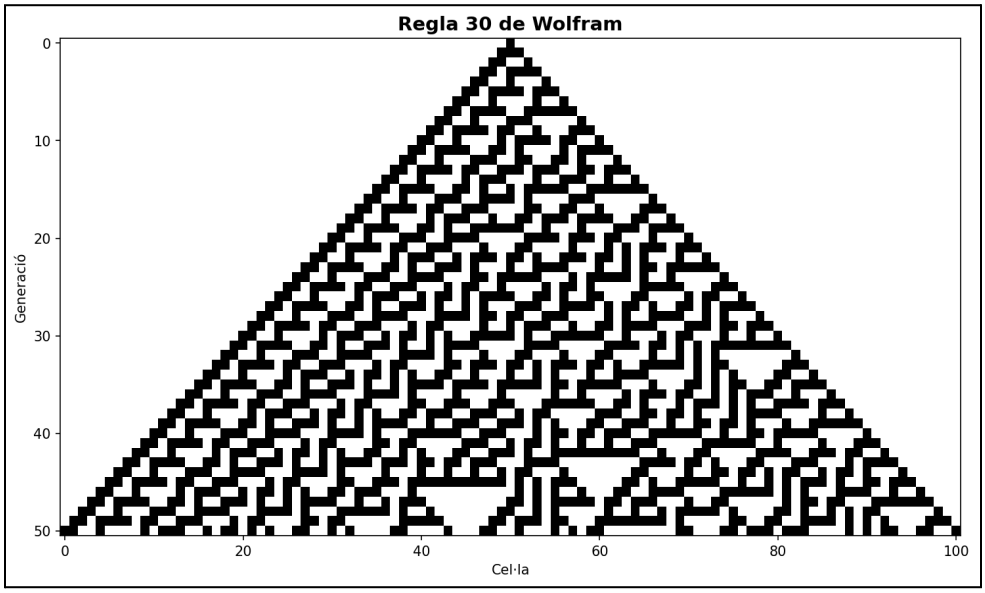


Figura 1: Visualització de la regla 30

Seguidament, hem comparat les regles 30 i 90 originals amb les dues variants de gra gruixut.

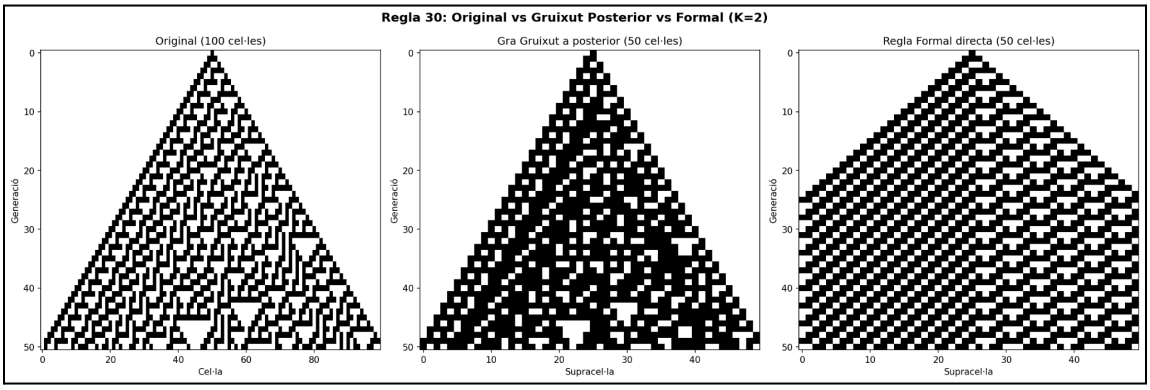


Figura 2: Comparació original vs gra gruixut: regla 30

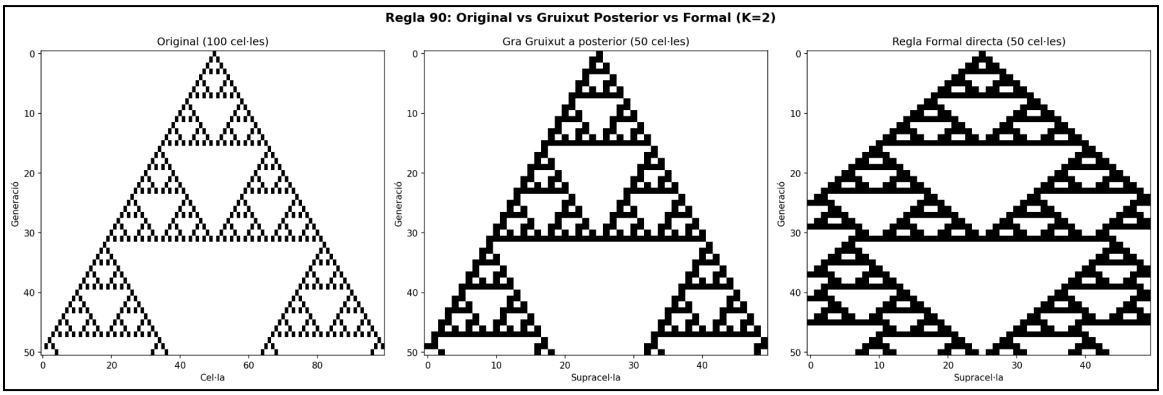


Figura 3: Comparació original vs gra gruixut: regla 90

Aqui podem veure un fenòmen curiós. Mentre que el gra gruixut a posteriori manté les proporcions de l'original perfectament, la derivació amb la nova regla formal es desvia una mica de l'esperat. Aquesta diferència es deu a que, mentre que el mètode a posteriori és una simplificació visual exacta que es construeix a partir d'una simulació prèvia, la regla formal és una aproximació que va acumulant error a cada pas (ja que no té una referència del que s'ha fet abans). Això porta a que l'estructura es desviï lleugerament. Tot i així, cal destacar que la derivació formal de la regla 90 s'ajusta bastant millor que la derivació formal de la regla 30.

Finalment, hem decidit mostrar diverses regles en una mateixa graella per veure alguns casos més (veure Annex) i, per acabar, hem combinat les regles 30 i 90.

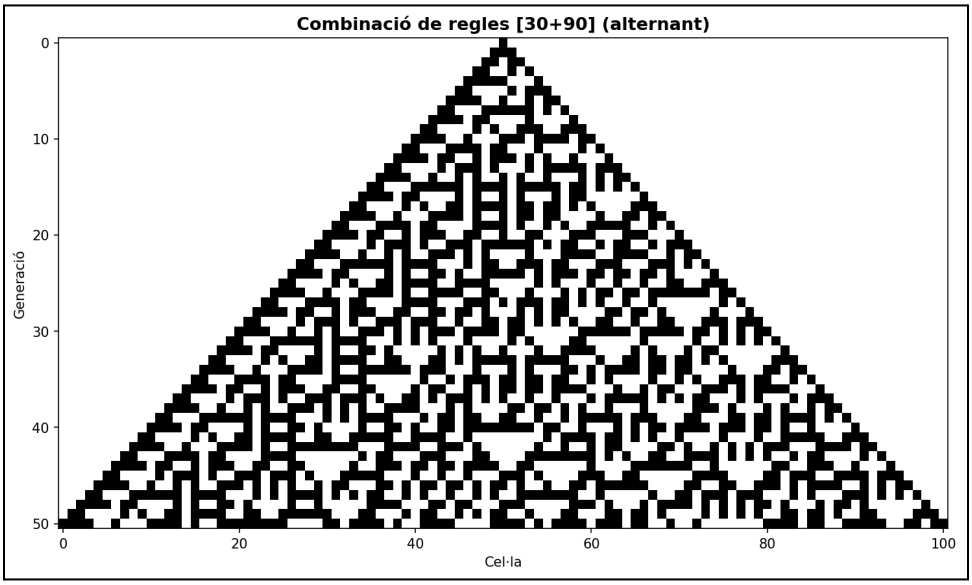


Figura 4: Combinació de regles 30 i 90

Conclusions

Aquesta primera sesió ens ha permès entendre com funcionen els autòmats cel·lulars de manera general, i com ho fan per generar les diferents regles. Hem explorat diversos casos i comprovat amb diferents proves que el funcionament del codi és correcte i adequat.

Això ens servirà com a transició natural cap als models m:n-CA^k, molt més complexos i els quals utilitzarem per la segona part d'aquesta pràctica: la modelització i evolució d'un incendi forestal.

Segona Sessió

Introducció i objectius

Durant la segona sessió, durem a terme una simulació d'un incendi forestal a partir de dades reals i del que hem fet durant la primera sessió. El que volem és ser capaços de modelar l'evolució de l'incendi i veure quines àrees seran afectades més ràpidament. Per fer-ho, definirem un model m:n-CA^k, el qual també representarem amb SDL.

Rebrem les dades a través de diferents fitxers que ens proporcionaran informació com el combustible, la humitat, la vegetació, etc. L'objectiu final és que el model treballi amb un cert nombre de capes que interactuïn entre elles i que, a partir d'aquestes, es pugui veure com evoluciona l'incendi.

Assumpcions del model

Per a fer aquest modelatge, hem hagut de fer una sèrie d'assumpcions per tal de simplificar el model i poder treballar-hi d'una manera més fàcil. A continuació, es mostra una llista amb les diferents assumpcions que hem fet:

- **El bosc té una àrea limitada:** definim una àrea en la qual treballarem (per exemple, 20x20) i només avaluarem l'evolució de l'incendi dins d'aquesta. Considerem que tot el que passi fora no existeix per a la simulació.

- **Sabem l'estat de cada cel·la en tot moment:** per tal de simplificar la representació de l'evolució de l'incendi, assumim que sabem quin és l'estat de cada cel·la en tot moment (cremant-se, cremada o bé sense cremar). També sabem la quantitat de

vegetació, vent, combustible, humitat i relleu en tot moment i aquesta és la mateixa per cada cel·la.

- **La quantitat de vent es manté constant:** assumim que durant tota la simulació la força i la direcció del vent es mantenen constants i que en cap moment apareixen ràfegues de vent ni aquesta disminueix.

- **Coneixem la manera en què es propaga el foc:** sabem com el foc es propaga entre cel·les veïnes (funció de veïnatge) i també assumim que, mentre hi hagi humitat en una cel·la, aquesta no es podrà cremar i que després es cremarà durant més o menys unitats de temps depenent de la vegetació que tingui. Aquest comportament és el mateix per a totes les cel·les.

- **Temps discret:** El temps avança en passos discrets (ticks) i en cada pas s'actualitza l'estat de totes les cel·les alhora.

- **Absencia d'intervenció humana:** No es tenen en compte accions externes com bombers, tallafocs artificials o extinció manual les quals podrien afectar l'evolució del incendi.

- **Ignició inicial definida:** L'incendi comença en un conjunt inicial de cel·les conegudes i no apareixen nous focus espontanis durant la simulació.

- **No hi ha regeneració:** La vegetació no es regenera durant el temps de la simulació, un cop s'acaba de cremar és queda així durant tota la simulació.

Capes del model

Per a fer aquest model treballarem amb un total de 6 capes, les tres que demanava l'enunciat: vegetació, humitat i foc i tres més que afegim nosaltres per tal de fer la simulació més fidedigne les quals són combustible, relleu i vent. En la següent taula podem veure una descripció de cada capa.

Nom	Tipus	Descripció	Qtt	Font	Mod?
Humitat	Raster	Temps que una cel·la pot estar sense cremar-se	1	Enunciat	Sí
Vegetació	Raster	Temps que una cel·la pot estar cremant-se	1	Enunciat	Sí
Foc	Raster	Estat de la cel·la (sense cremar, cremant-se, cremada)	1	Model	Sí
Relleu	Raster	Influència de l'orografia en la propagació del foc	1	IA	No
Combustible	Raster	Quantitat de combustible disponible per cel·la	1	IA	Sí
Vent	Vectorial	Direcció i intensitat amb tendència de propagació del foc	1	IA	No

Taula 1: Descripció de les capes del model

Podem veure com es defineix cada capa: una petita descripció, la quantitat que en tenim, la seva font (les que no disposàvem de l'enunciat les vam extreure de la IA assegurant-nos que seguissin un format correcte) i, finalment, si es poden modificar o no.

Model SDL

Tal com hem mencionat abans, aquest automat cel·lular es pot expressar en forma d'un model de SDL. A continuació farem una petita descripció dels estats escollits i dels seus processos, la resta es podran veure als annexos.

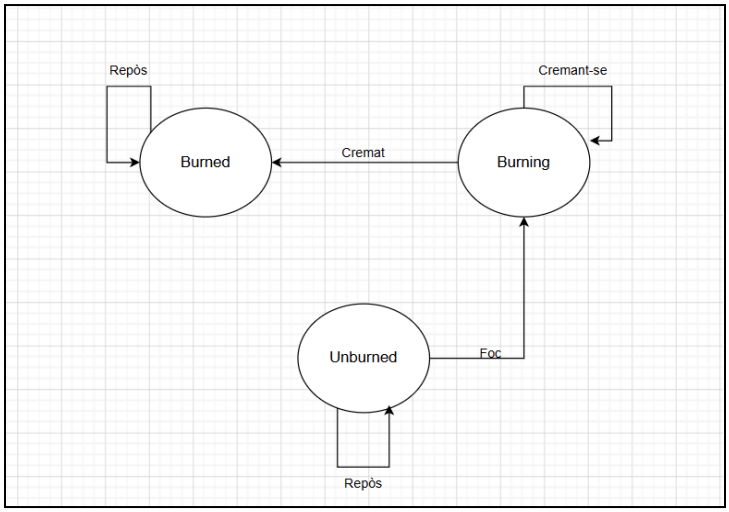


Figura 5: Descripció dels estats en SDL

Tal com podem veure en la primera figura, el nostre model disposa de tres estats per a cada cel·la. El primer, "**Unburned**", fa referència a quan una cel·la es troba en un estat normal, és a dir, no ha entrat en contacte amb el foc encara. Estant en aquest estat pot rebre dos senyals: o bé *repòs*, que significa que no té foc en cap dels seus veïns, o bé *foc*, que simbolitza que ja ha entrat en contacte amb el foc i que se li ha acabat la humitat.

Un cop entra en contacte amb el foc, passa a l'estat "**Burning**". En aquest estat, mentre encara tingui vegetació, seguirà *cremant-se* fins que aquesta vegetació arribi a 0 i rebi el senyal *cremat*, cosa que farà que passi a estar a l'estat "**Burned**", on es mantindrà en *repòs* fins que s'acabi la simulació.

Estat Inicial

A continuació, descriurem la definició de l'estat inicial per cada cel·la per a veure en quin estat es defineix quan comença la simulació.

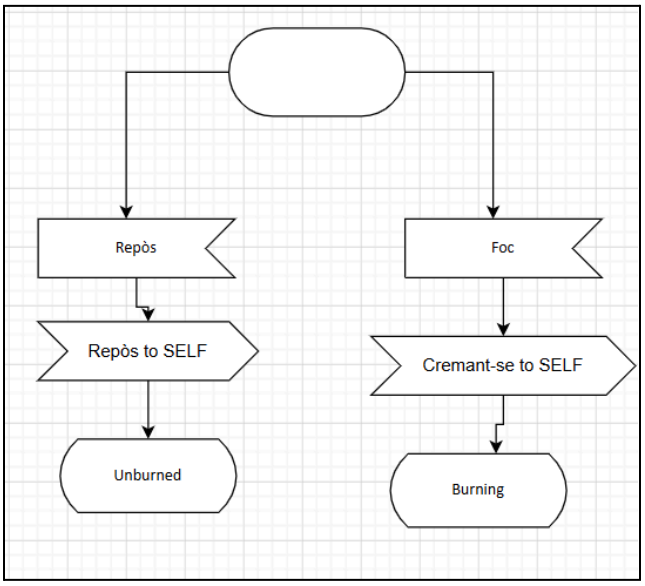


Figura 6: Model SDL de l'estat inicial

Tal com podem veure, la definició de l'estat inicial és molt senzilla, depenent del senyal que rebi cada cel·la, la qual es defineix en l'inicialització del model, pot inicialitzarse com 'Unburned' si rep repòs o bé com 'Burning' si rep foc, enviant el senyal corresponent depenent de cada cas.

Estat Unburned

A continuació, descriurem la definició de l'estat Unburned per cada cel·la:

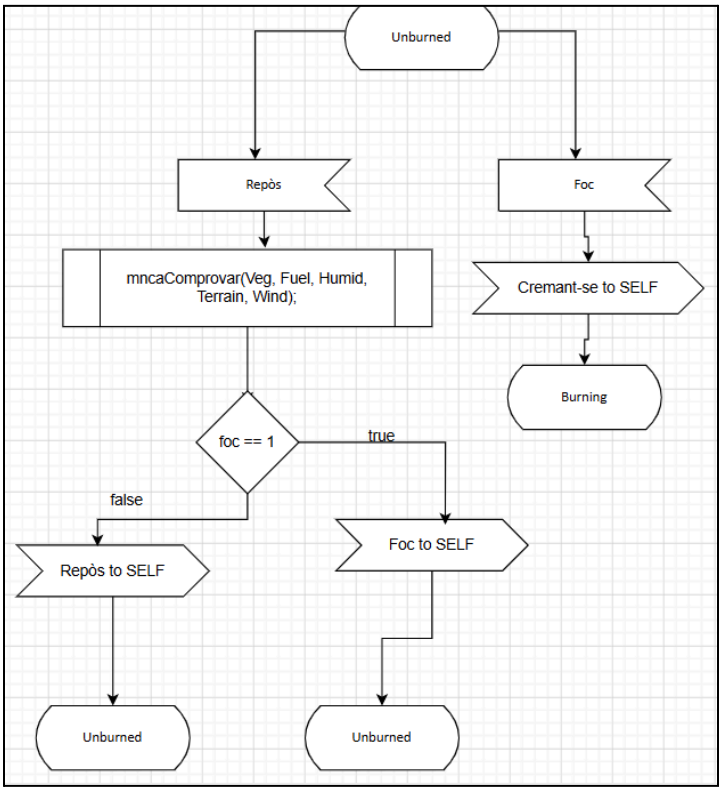


Figura 7: Model SDL de l'estat Unburned

Podem veure com l'estat **"Unburned"** es defineix seguint les mateixes lògiques que portem descrivint. Pot rebre o bé *foc* o bé *repòs*; si rep *foc*, passa a l'estat **"Burning"**, i si rep *repòs*, crida a la funció **mncaComprovar**, la qual és una simplificació del codi que ens indica si s'està cremant tenint en compte tant la seva pròpia situació (humitat i combustible) com la dels veïns. Si s'està cremant, envia el senyal *foc* a si mateix i, si no, envia el senyal *repòs*.

Estat Burning

A continuació, descriurem la definició de l'estat Burning per cada cel·la:

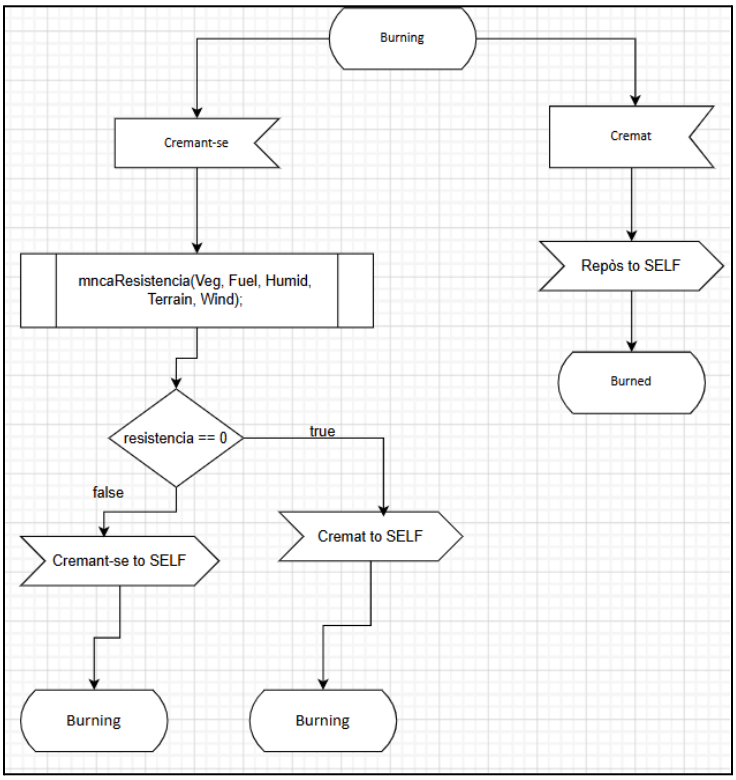


Figura 8: Model SDL de l'estat Burning

Podem veure com, en aquest cas, també es tracta d'un procés molt senzill. Si s'està cremant, crida a una funció que calcula si encara li queda vegetació per cremar i, si és així, segueix cremant-se; en cas contrari, envia el senyal *cremat* a si mateix i després passa a l'estat **"Burned"** amb el senyal de *repòs*.

Estat Burned

Aquest estat és el més senzill de tots, ja que senzillament està en un bucle infinit on sempre es manté en repòs. Considerem que, si una cel·la està cremada, es quedarà així durant tota la simulació.

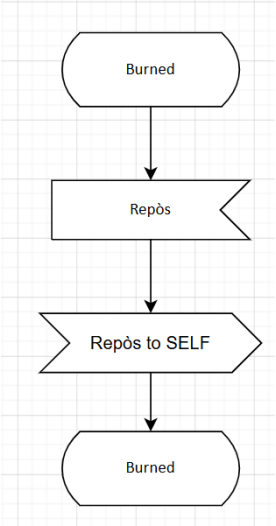


Figura 9: Model SDL de l'estat Burned

Modelització en Python

En aquest apartat discutirem les parts més importants del codi i com s'ha decidit implementar aquest model en Python. Tot el codi està comentat, així que no farà falta entrar massa en detall en què fa cada funció i cada línia. Senzillament, ens centrarem en el que considerem més important.

La part més important de la lògica del model està definida en la classe WildfireCA, aquí és on defineix la propagació del incendi i es calculen els factors que poden fer que aquest vagi més ràpid, més lent o prengui certes direccions més favorables.

Dins d'aquesta classe es defineixen les diferents capes del model (humitat, vegetació i estat del foc), així com altres capes auxiliars com el relleu, el combustible i el vent, que actuen com a moduladors del comportament del sistema. La propagació del foc es basa en un veïnatge de Moore (8 veïns), de manera que una cel·la pot veure's afectada pel foc si algun dels seus veïns està cremant-se.

La funció més rellevant és la funció d'evolució (step), que actualitza l'estat de totes les cel·les a cada instant de temps. El model segueix un esquema d'actualització síncrona, en què a cada iteració es recorren totes les cel·les de la graella i es calcula el nou estat en funció de l'estat actual i dels seus veïns. Per evitar interferències durant l'actualització, es treballa amb còpies temporals de les capes (com new_state, new_veg o new_hum), de manera que tots els canvis es calculen a partir de l'estat anterior i s'apliquen simultàniament al final de cada iteració.

En aquesta funció es defineixen les transicions entre els tres estats possibles: Unburned, Burning i Burned. Una cel·la passa de no cremada a cremant-se quan rep influència d'algun veí en flames i supera un cert llindar determinat per la humitat, introduint així un retard en la propagació del foc. Un cop en estat de combustió, la vegetació es va consumint progressivament fins que arriba a zero, moment en què la cel·la passa a l'estat cremat, que és un estat final.

A més, el model incorpora diversos factors que modulen la propagació del foc. El vent introdueix un biaix direccional que afavoreix la propagació en certes direccions, el relleu té en compte la pendent del terreny (afavorint la propagació en pujada i dificultant-la en baixada) i el combustible determina la intensitat amb què el foc pot avançar en cada cel·la. Aquests factors es combinen en un únic factor de propagació que afecta el temps necessari perquè una cel·la s'encengui.

Finalment, el model inclou funcions auxiliars com la definició del veïnatge o la inicialització del foc, així com un mètode d'execució (run) que permet simular l'evolució de l'incendi al llarg del temps fins que no queden cel·les en combustió o s'arriba a un nombre màxim d'iteracions.

Resultats

El codi retorna un total de sis gràfics, els quals faciliten la interpretació del model i permeten visualitzar l'avenç de l'incendi a través del temps. Per simplificar l'informe, posarem tots els gràfics a l'apartat d'Annex (amb l'excepció del gràfic de l'evolució de l'incendi).

El primer gràfic mostra les capes abans d'iniciar el model, per tal de poder fer-nos una idea de, per exemple, quines cel·les tenen més o menys vegetació. El segon, que es pot veure a continuació en aquest mateix apartat, mostra l'avenç del foc a mesura que passa el temps.

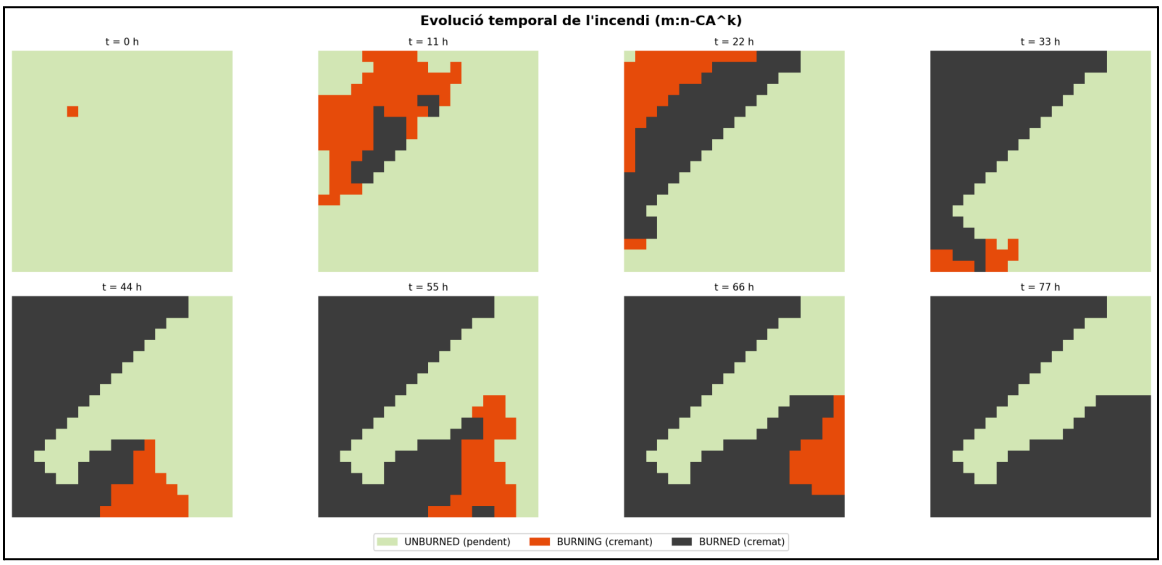


Figura 10: Evolució de l'incendi a mesura que avança el temps

Podem veure com la figura ens mostra com evoluciona el temps; si tenim en compte tant les capes de vegetació, humitat i l'efecte del vent (veure /output a l'entregable), veiem que el resultat té tot el sentit, ja que al principi el foc es propaga cap a zones amb menys humitat i més vegetació, tot seguint la direcció del vent. A més, també és coherent amb les capes de combustible i relleu.

El tercer gràfic mostra quantes cel·les hi ha en cada estat per a cada instant de temps; es pot observar que, en finalitzar la simulació, no queda cap cel·la cremant-se. El quart és una comparativa entre l'estat final i les capes inicials, on es pot veure com, en zones amb més humitat i menys vegetació, l'incendi no hi arriba. Aquestes també es podràn veure al annex.

Finalment, les dues últimes gràfiques mostren l'efecte del vent en el model i què passaria si aquest es propagués en altres direccions. La direcció escollida en el nostre model ha estat la del sud-oest.

Conclusions

Aquesta pràctica ens ha permès veure com l'autòmat cel·lular que vam treballar a classe i que vam implementar durant la primera part pot tenir aplicació en un cas real, com ara la simulació d'un incendi forestal. Hem treballat amb diferents capes i hem observat com aquestes interactuen entre elles per reproduir un comportament el més realista possible, tot mantenint certes assumpcions que han permès simplificar el model.

En definitiva, aquesta pràctica ens ha ajudat a entendre com un autòmat cel·lular pot aplicar-se a un problema real i com pot ser una eina útil per simular fenòmens complexos com un incendi forestal. A partir d'aquest tipus de models, també es pot millorar la capacitat d'anàlisi i preparació davant situacions reals d'aquest tipus.

Ús de la IA

Durant el transcurs de la pràctica s'ha fet ús de la IA en diversos apartats. El primer ha estat la part de codi: degut a la gran quantitat de codi que havíem de desenvolupar i al temps limitat de què disposàvem, vam decidir utilitzar la IA per ajudar-nos a organitzar idees i a generar part del codi, el qual posteriorment hem revisat per assegurar-nos que l'enteníem completament i que funcionava correctament. Un cop fetes les correccions pertinents sobre el codi generat, vam comprovar el seu funcionament i vam afegir alguns gràfics que inicialment no teníem.

D'altra banda, la IA també ens ha ajudat a estructurar l'informe. Tot i que el redactat l'hem realitzat nosaltres, també l'hem utilitzat per corregir errors ortogràfics i per millorar la claredat i la comprensió d'alguns apartats.

Seguint la proposta de la CETRA, considerem que hem fet un ús correcte de la IA, ja que hem respectat les 10 regles que es proposen en el decàleg. Per començar, com que les dades eren generades i no treballàvem amb dades reals, no hem hagut d'utilitzar cap IA en específic, ja que no hi havia problemes de confidencialitat. També hem supervisat tot el que ha generat la IA i, finalment, n'hem comunicat l'ús i assumim la responsabilitat total de qualsevol error en el codi, ja que nosaltres mateixos l'hem revisat i acceptat.

Les IA utilitzades han estat principalment Claude per a la generació de codi, ChatGPT per a la correcció i revisió del text, i Gemini en menor mesura per aclarir alguns conceptes teòrics que no quedaven del tot clars.

Annexos

Primera Sessió

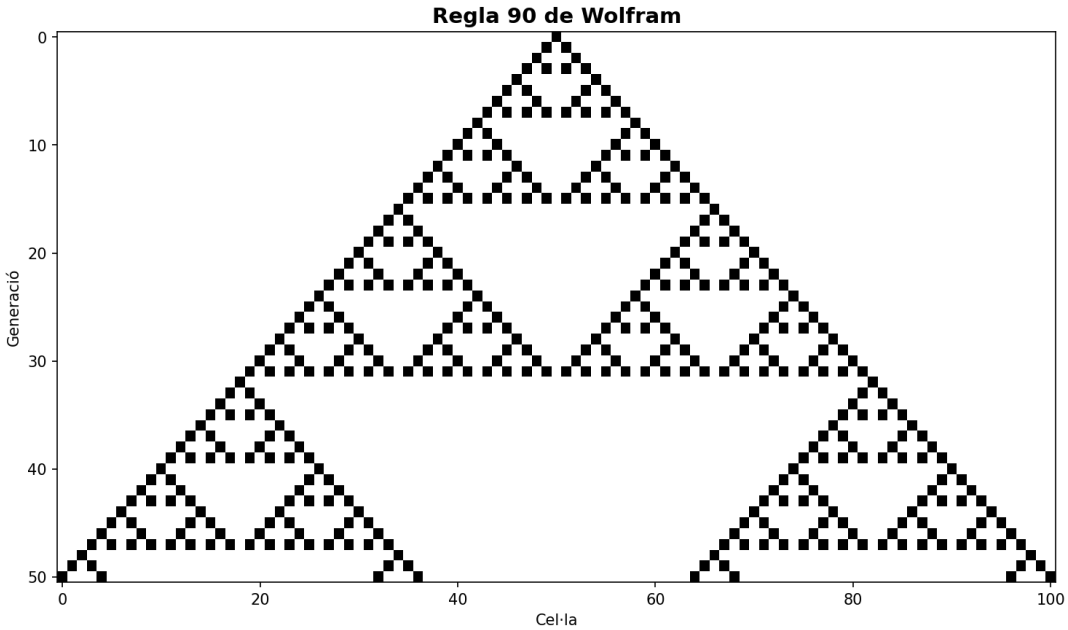


Figura 11: Visualització de la regla 90

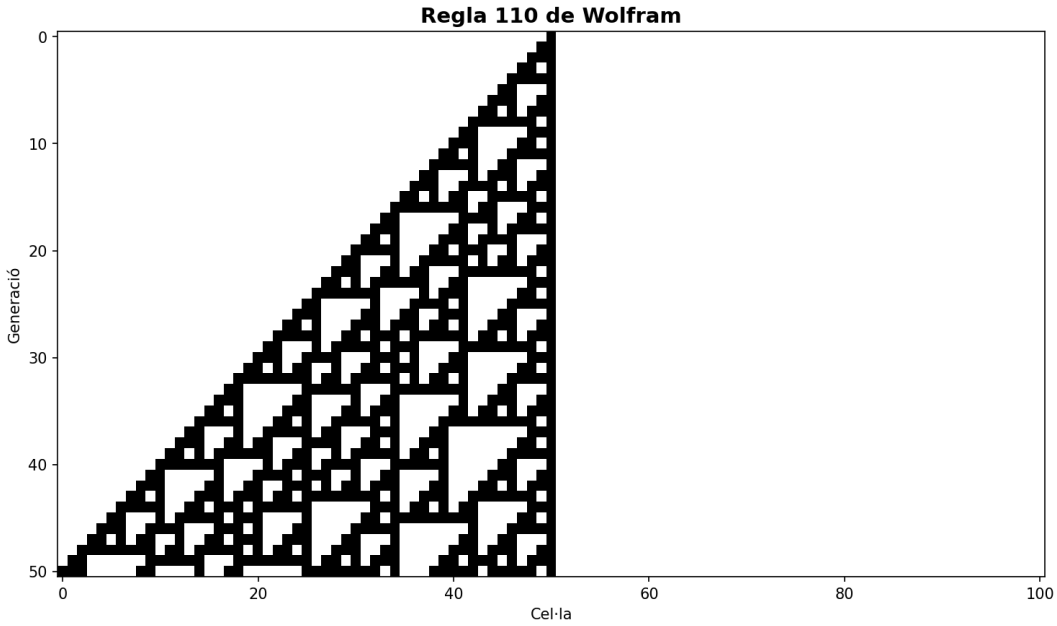


Figura 12: Visualització de la regla 110

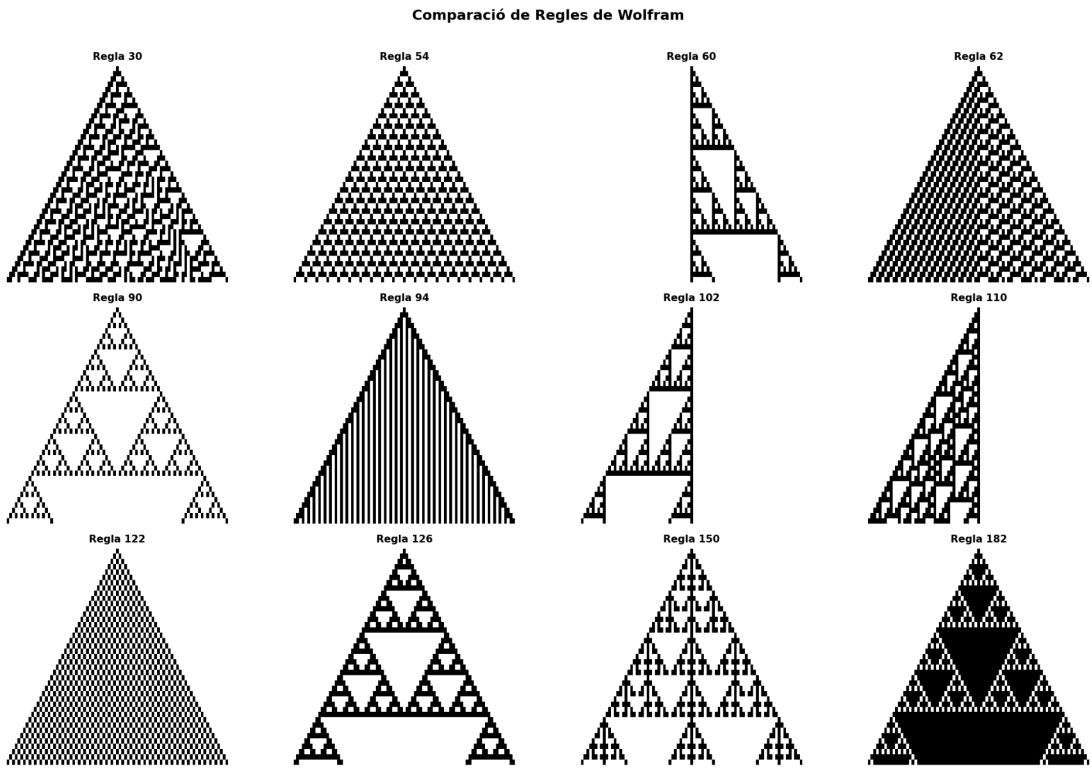


Figura 13: Visualització de múltiples regles

Segona Sessió

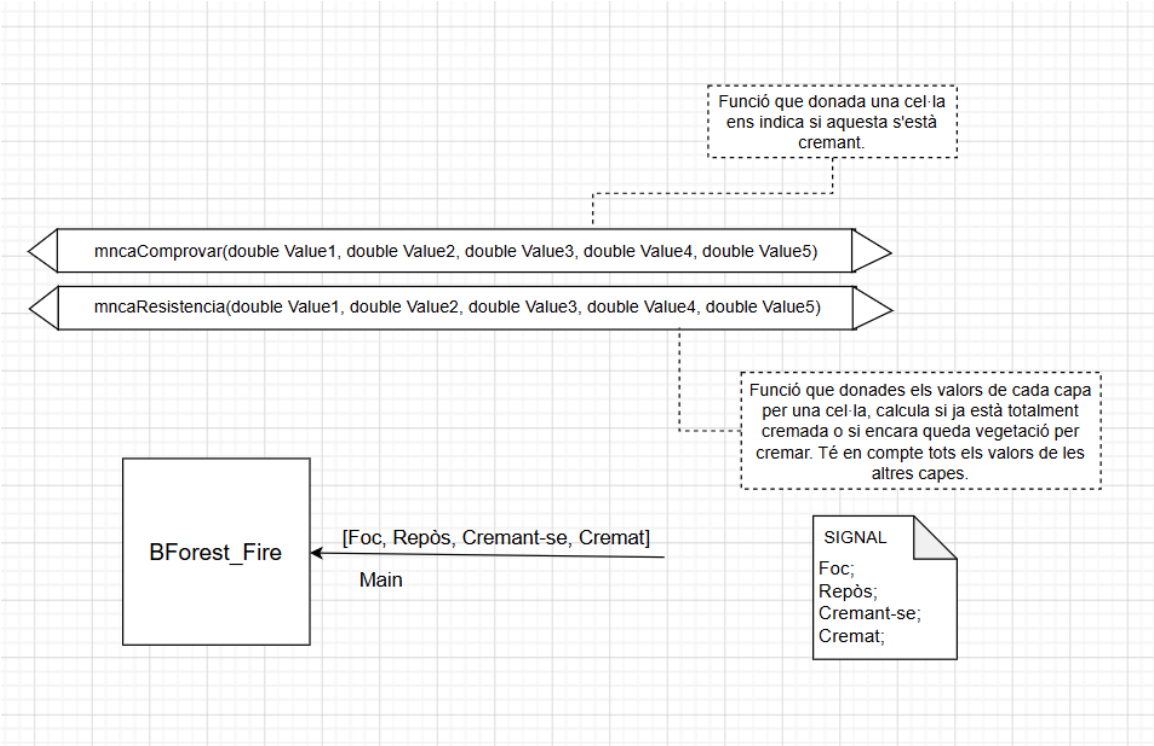


Figura 14: Sistema MNCA_Forest_Fire

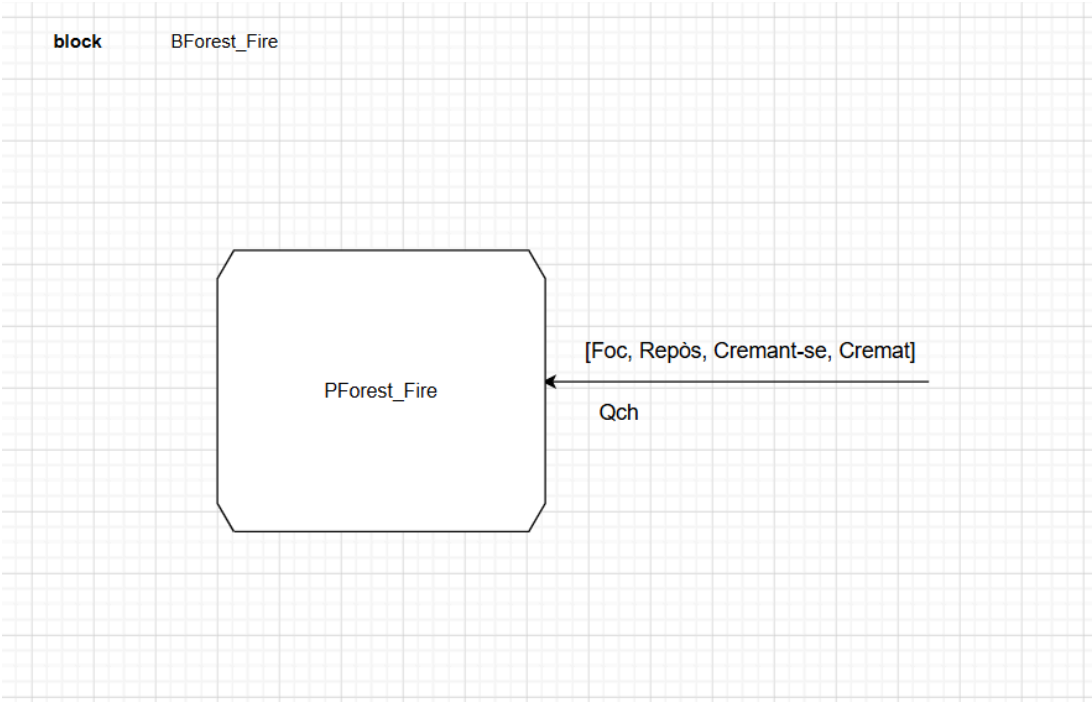


Figura 15: Bloc de Forest_Fire

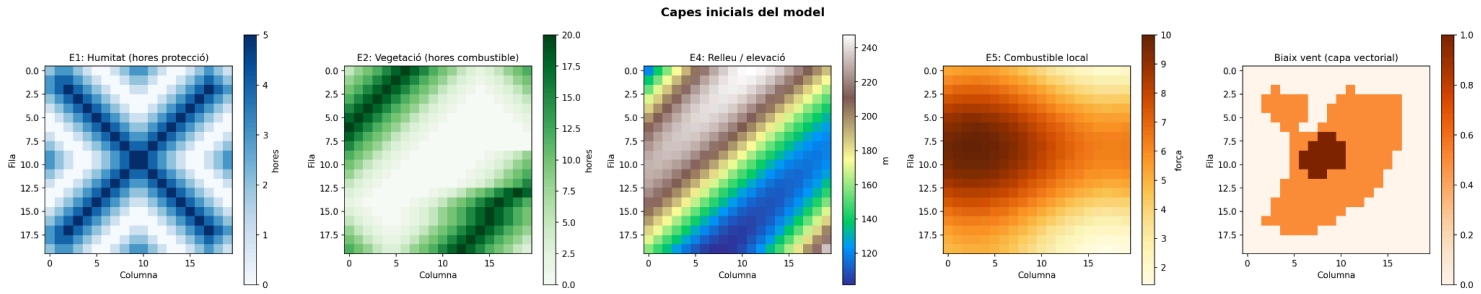


Figura 16: Capes inicials del model

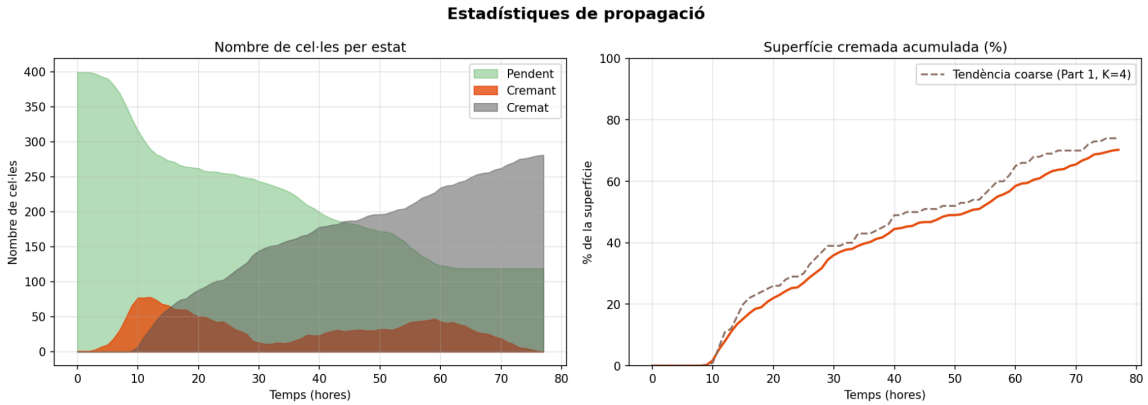


Figura 17: Comparació de cel·les segons l'estat

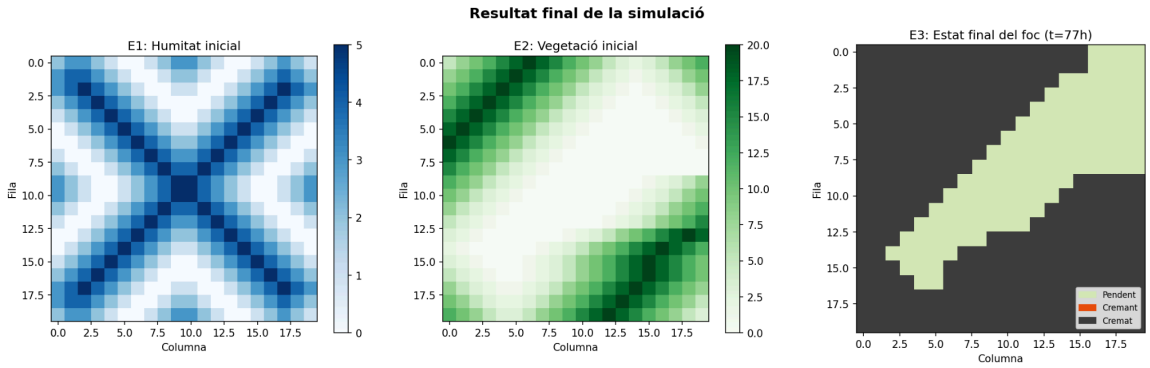


Figura 18: Comparativa entre l'estat inicial i el final

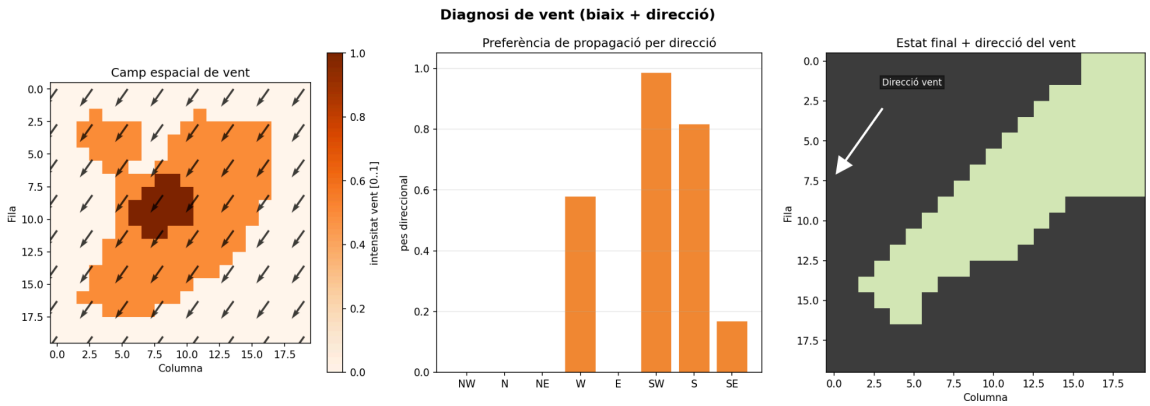


Figura 19: Descripció i direcció de la capa vectorial del vent

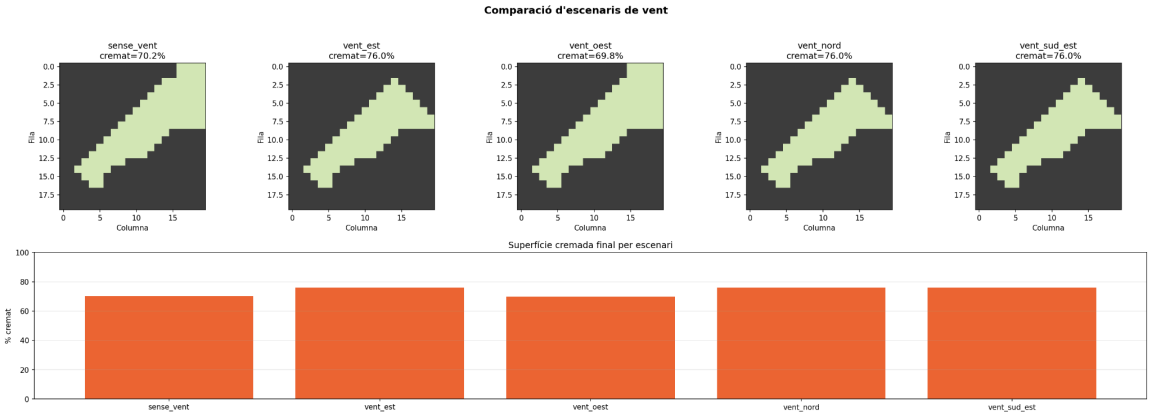


Figura 20: Comparativa de resultats segons direcció del vent